

Rooting for Kepler: Testing, Explaining and Repairing With-Memory Root Finders on Kepler's Equation

CSE 402 Final Project Report | Section A, Group 1

Anisa Binte Asad
Student ID 2105036, BUET
Dhaka, Bangladesh

Dipit Saha
Student ID 2105050, BUET
Dhaka, Bangladesh

Atika Tabassum Suchi
Student ID 2105053, BUET
Dhaka, Bangladesh

S. M. A. M. Mahdi
Student ID 2105056, BUET
Dhaka, Bangladesh

Fariha Ifrat
Student ID 2105059, BUET
Dhaka, Bangladesh

Abstract

Kepler's equation, $E - e \sin E = M$, is solved every time an orbit is evaluated, about 130,000 times in one radial-velocity fit. Two recent papers propose with-memory iterative methods, NWM9 (2024) and NWM11 (2025), that reach convergence orders 8.90 and 10.74 with no extra function evaluations, but neither tests them on Kepler's equation. We reimplemented both, confirmed their orders at 2000 digits, and compared them with Newton's, Danby's and Markley's methods in 125,800 solves at 7,400 pairs of eccentricity e and mean anomaly M , from four starting guesses. The new methods need the fewest iterations (NWM11 1.38, Danby 1.94). In our cost model their median solve is 17% cheaper than Danby's, owing to the eighth-order step rather than the memory, but in Python it takes 1.3 to 1.8 times as long. In the corner $e > 0.9$, $M < 0.1$ they fail 33 to 37 times as often as Danby. The cause is the self-accelerating parameter, which divides by $f'(E) = 1 - e \cos E$, so near $f' = 0$ its correction grows as large as the derivative it adjusts. We propose a one-line safeguard that keeps the parameter only when its correction is at most half of $|f'|$. Safeguarded NWM11 cuts its failures from 74 to 3 (Danby has 2) and keeps its iteration count and its order of 10.74 up to $e = 0.999$. The starting guess matters as much as the method: Napier's guess lowers NWM9's mean iteration count from 2.37 to 1.44. Even in the worst case, solver error moves the fitted orbit about 1,500 times less than measurement noise does.

Keywords

Kepler's equation, with-memory iterative methods, convergence order, root finding, safeguarding, radial velocity, benchmarking

1 Introduction and Base Paper Review

1.1 The problem

A planet's position along its orbit is described by the mean anomaly M , which grows linearly with time, and the eccentric anomaly E , which fixes where the planet actually is. The two are linked by Kepler's equation

$$f(E) = E - e \sin E - M = 0, \quad 0 \leq e < 1, \quad (1)$$

where e is the orbital eccentricity. The equation has no closed-form solution, so it is solved numerically every time an orbit is evaluated. RadVel [1], a standard toolkit for fitting exoplanet orbits to radial-velocity data, solves it once per observation for every likelihood evaluation. In our experiments a single maximum-likelihood fit in

RadVel needed about 1.3×10^5 solves (Section 4.6), and a Markov chain Monte Carlo run evaluates the likelihood many more times than one fit does. The speed and reliability of the solver therefore matter in practice.

Two properties of Equation (1) matter for the rest of this report. First, its derivative $f'(E) = 1 - e \cos E$ approaches zero as $e \rightarrow 1$ and $M \rightarrow 0$, because the root E is then close to zero and $\cos E \approx 1$. Every Newton-type step divides by f' , so this corner of the (e, M) plane is where solvers struggle. Second, f and f' are built from $\sin E$ and $\cos E$ at the same point, so a method that needs both pays for one pair of trigonometric calls, not two separate evaluations. A method's cost on this equation is therefore not the same as its cost on a generic nonlinear equation.

RadVel's own solver is Danby's fourth-order method [4], written in C following Murray and Dermott [3], with a residual tolerance of 10^{-12} [2]. That is the method a new solver has to beat.

1.2 The base papers

Our base papers are two articles by Mittal, Panday and co-authors [6, 7]. Both start from an optimal eighth-order, three-step method and raise its order with *self-accelerating parameters*: quantities that are recomputed every iteration from points the method has already evaluated. Because the method reuses information from earlier iterations, it is called a *with-memory* method.

NWM9 [6] adds one parameter, α_n , to the third step of the eighth-order method of Matthies et al. [8]. The paper proves an R-order of at least 8.8989 and an efficiency index that rises from 1.6818 to 1.7272.

NWM11 [7] adds two parameters, α_k and β_k , to the first and third steps of the eighth-order method of Solaiman and Hashim [9]. The paper proves an R-order of at least 10.7446 and an efficiency index that rises from 1.6818 to 1.8105.

In both methods the parameters are computed from derivatives of Hermite interpolating polynomials fitted through the current and previous iterates. Those points have already been evaluated, so the higher order costs no new function evaluations. That is the central claim of both papers.

1.3 What the base papers leave open

Both papers test their methods on eight smooth scalar functions, each from a single starting point, and measure cost with the efficiency index $\rho^{1/\gamma}$, where ρ is the order and γ counts the function

and derivative evaluations per iteration. This leaves four questions open for Kepler’s equation.

- (1) **Cost.** The efficiency index charges f and f' as two evaluations and an interpolated derivative as none. On Kepler’s equation f and f' come from one (sin, cos) pair, and in our Python code an interpolated derivative costs more than a sin call.
- (2) **Behaviour across the domain.** Eight test functions say nothing about how a method behaves across the whole (e, M) plane, and in particular near $e \rightarrow 1$, $M \rightarrow 0$ where $f' \rightarrow 0$.
- (3) **Robustness.** Neither paper reports how often the methods fail to converge, or why.
- (4) **Relevance.** Neither paper asks whether a higher order changes anything in an application whose data are noisy.

1.4 Our contributions

Our objective is to test whether the higher order of NWM9 and NWM11 pays off on Kepler’s equation, and to repair the methods where they fail. We keep the published formulas exactly as given and change them only after finding where they break. Our contributions are:

- (1) **A verified reimplement.** Both methods reproduce their published orders at 2000-digit precision. We also show that matching the order is not enough to prove a correct transcription, and that each paper’s error constant is the check that works (Section 4.1).
- (2) **A Kepler-specific benchmark** of five solvers and four starting guesses over 125,800 solves, with cost priced in measured sin-call units (Section 4.2). It shows that the with-memory methods’ median saving over Danby comes from the eighth-order step, not the memory, and that the starting guess can matter more than the method: replacing $E_0 = M$ with Napier’s guess cuts NWM9’s mean iteration count from 2.37 to 1.44 and removes all of its failures (Section 4.3).
- (3) **An explanation of the failures.** The self-accelerating parameter divides by f' , so near $f' = 0$ its correction can grow as large as the derivative it adjusts. Switching the memory off shows that the parameter is the cause: all 74 of NWM11’s failed solves and 79 of NWM9’s 81 then converge (Section 4.4).
- (4) **An improvement to the base method.** A safeguard on the parameter that cuts NWM11’s failures from 74 to 3, against Danby’s 2, without changing its order or its iteration count (Section 4.5).
- (5) **A downstream check** showing how far solver error travels into a fitted orbit, compared with the noise in real data (Section 4.6).

All code, configuration files, result tables and figure scripts are public at <https://github.com/kittocat1/CSE402-Rooting-for-Kepler->.

2 Methodology

Figure 1 shows how the study fits together. This section describes the methods and how we measure them; Section 3.1 onward describes the experiments.

2.1 The with-memory iteration

We use NWM11 to show the structure; NWM9 is built the same way with one parameter instead of two. Starting from s_k , one NWM11 iteration is

$$v_k = s_k - \frac{f(s_k)}{f'(s_k) + \alpha_k f(s_k)}, \quad (2)$$

$$t_k = v_k - \frac{f(v_k)}{q} - \frac{2f(v_k)^2 q R}{4q^4 - 4f(v_k)q^2 R + f(v_k)^2 R^2},$$

$$s_{k+1} = t_k - \frac{f(t_k)}{w' + \beta_k f(t_k)}, \quad (3)$$

where q , R and w' are divided-difference estimates of $f'(v_k)$, $f''(v_k)$ and $f'(t_k)$ built from values already in hand [7]. The parameters are $\alpha_k = -H_5''(s_k)/(2f'(s_k))$ and $\beta_k = H_7''''(t_k)/(4H_6''''(v_k)) + \alpha_k$, where H_5 , H_6 and H_7 are Hermite polynomials through the current and previous iterates. With $\alpha_k = \beta_k = 0$ the scheme is the eighth-order method of Solaiman and Hashim. Each iteration evaluates f at three points but needs f' only at s_k , so it uses one (sin, cos) pair and two lone sin calls.

Two implementation details decide whether the parameters come out right. The node lists repeat a point, for example

$$[s_k, s_k, t_{k-1}, v_{k-1}, s_{k-1}, s_{k-1}],$$

which makes an ordinary divided difference 0/0. Our interpolation code therefore uses the confluent (Hermite) form and takes f' at a repeated node. The papers also need the derivative $H^{(j)}$ itself, not the Taylor coefficient $H^{(j)}/j!$. Using the coefficient divides each derivative by $j!$ and so gives wrong parameters, yet the method still converges, so the mistake would not show up as a crash.

2.2 Measuring convergence order

We check each claimed order instead of assuming it, using the computational order of convergence

$$p \approx \frac{\ln |\varepsilon_{n+1}/\varepsilon_n|}{\ln |\varepsilon_n/\varepsilon_{n-1}|}, \quad \varepsilon_n = |x_n - x^*|, \quad (4)$$

which needs three consecutive errors that are still above the precision floor. A method of order 9 starting at an error of 10^{-1} reaches 10^{-9} , then 10^{-81} , then 10^{-729} . In double precision only two of those errors are visible, which is not enough for one estimate. We therefore measure order at 2000 decimal digits with mpmath [16]; the same solver code runs on both floats and mpmath numbers. Everywhere else the solvers run in ordinary double precision.

2.3 Counting cost on Kepler’s equation

Counting iterations is unfair, because one NWM11 iteration does far more work than one Newton iteration. The efficiency index misprices that work, as noted in the introduction. Every solver in our code evaluates f through a shared problem object that records what each call needed. We then price each kind of call in units of one sin call, timed on the benchmark machine (Table 1).

We report two costings. The *measured* weights are what our Python code actually pays. The *call-count* weights describe a compiled implementation with a fused sincos routine, where a pair costs one call and arithmetic is nearly free. Like the efficiency index, this costing treats an interpolated derivative as free. Unlike

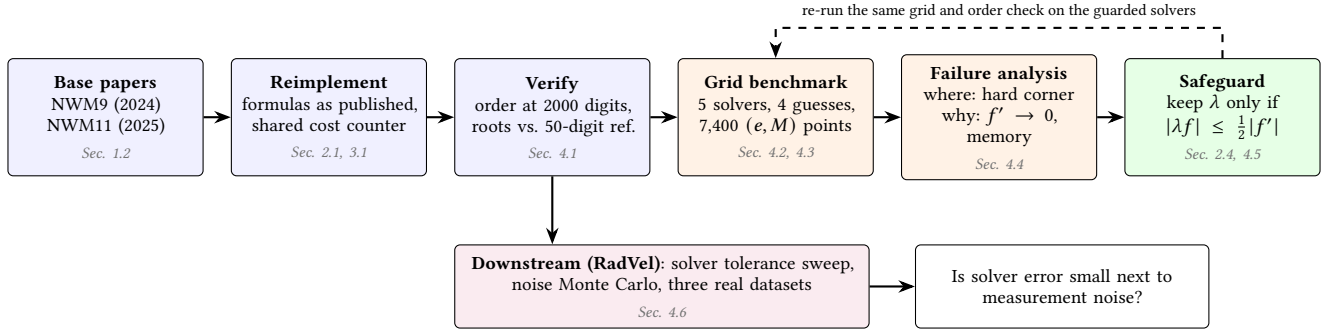


Figure 1: How the study is organised. Each box writes its results under results/ in the repository; the grey label gives the section that reports them.

Table 1: Cost of each primitive, in units of one sin call. The measured weights are the median of three timing runs. Over all repeats the pair ranged from 2.12 to 2.42 and the interpolated derivative from 1.66 to 1.86.

Primitive	Measured (Python)	Call count
sin alone	1.00	1
cos alone	1.04	1
(sin, cos) pair	2.34	1
Interpolated derivative	1.76	0

it, it charges f and f' at one point as a single call. The choice between them changes how the classical and with-memory methods compare, so we show both.

2.4 A safeguard for the self-accelerating parameter

Each parameter enters a denominator as a correction added to a derivative: $f'(s_k) + \alpha_k f(s_k)$ in Equation (2), and $w' + \beta_k f(t_k)$ in Equation (3). The papers derive the parameters assuming the iterate is already close to the root. There f is tiny and the correction is a small adjustment. Far from the root, or where f' is itself close to zero, the correction can be as large as the derivative it is meant to adjust, and the step is thrown far off. Near $f' = 0$ the parameter makes this worse, because it has f' in its own denominator. In NWM11, $\alpha_k = -H_5''(s_k)/(2f'(s_k))$, so the correction measured against the derivative is

$$\frac{|\alpha_k f(s_k)|}{|f'(s_k)|} = \frac{|H_5''(s_k) f(s_k)|}{2f'(s_k)^2}.$$

This ratio grows like $1/f'^2$ as $f' \rightarrow 0$, while the Newton step f/f' grows only like $1/f'$. NWM9’s α_n contains the same term $-H_5''/(2f')$ [6].

Our safeguard tests this directly. In each iteration, a parameter λ (standing for α_k , β_k or NWM9’s α_n) is used only if

$$|\lambda f| \leq c |f'(s_k)|, \quad c = \frac{1}{2}, \quad (5)$$

where f is the function value the parameter multiplies and $f'(s_k)$ is the one derivative the iteration actually evaluates. We use $f'(s_k)$ for β_k too, because w' estimates the same derivative at a nearby point.

Once the memory is in use, the test for NWM11’s α_k reads exactly $|H_5''(s_k) f(s_k)| \leq f'(s_k)^2$, which holds near the root, as the papers’ derivation assumes. If the test fails, that parameter is set to zero for that iteration, so that sub-step is the same as in the memoryless eighth-order method. The test costs two multiplications and no new evaluations.

The safeguard cannot lower the order. Near a simple root, $f \rightarrow 0$ while $f'(s_k) \rightarrow f'(E^*) \neq 0$, and the parameters stay bounded, so after some iteration $|\lambda f|$ stays below $c|f'|$ and the test passes every time. From then on the guarded method takes exactly the published steps, so it has the published asymptotic order. We confirm this numerically in Section 4.5. For α_k , $c = 1/2$ keeps the denominator $f'(s_k) + \alpha_k f(s_k)$ between $\frac{1}{2}$ and $\frac{3}{2}$ times $f'(s_k)$, with the same sign, so the parameter can at most double the Newton step and can never reverse it. Every number in this report uses $c = 1/2$. Figure 2 shows where the two tests sit in one guarded NWM11 iteration.

2.5 Starting guesses

Every iterative method needs a first estimate E_0 . Both base papers prove their orders only for a start close enough to the root [6, 7], and each of their test functions starts from one fixed point close to its root. On Kepler’s equation the guess decides how near the start is, so we treat it as a separate factor and test four: $E_0 = M$ (*simple*); $E_0 = M + e \sin M$ (*canonical*, the first term of the series solution); $E_0 = M + 0.85 e \operatorname{sign}(\sin M)$ (*radvel*, the guess in RadVel’s code [2]); and $E_0 = e \sin M + \max\{M, e(\sin M + 0.591)\}$ (*napier*, found by symbolic regression [10]). Markley’s method [5] is a closed-form starter followed by one fixed correction, so it takes no guess and runs once per point.

3 Implementation and Experiments

3.1 Code structure

The code is a Python package, `keplerbench`, built on NumPy [17], SciPy [18], pandas [19] and Matplotlib [20]; the radial-velocity study uses RadVel [1], whose MCMC runs on emcee [21]. Every solver and every starting guess implements one shared interface, so the experiment runner treats them all the same way and every iterative solver uses the same stopping rule: stop when $|f(E)| \leq 10^{-14}$, or give up after 50 iterations. Solvers never call `math.sin` directly. They evaluate Equation (1) through a `KeplerProblem` object that

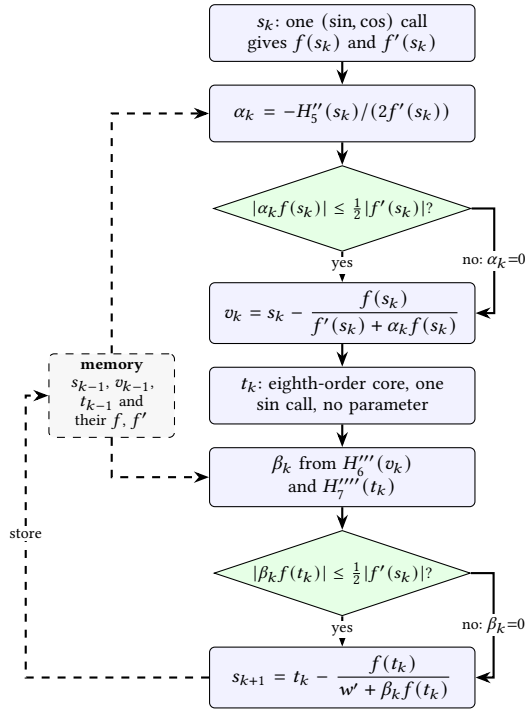


Figure 2: One iteration of guarded NWM11. The published method is the same diagram without the two green tests. In the first iteration there is no memory yet, so α_k and β_k take the papers’ fixed starting values.

counts every (sin, cos) pair and every lone sin or cos. Because this counting happens outside the solver, a solver cannot under-report its trigonometric work. The with-memory solvers also log each interpolated derivative on the same counter. The cost of a solve leaves out the starting guess, which costs one sin M call for every guess except $E_0 = M$. Table 2 lists the solvers.

Table 2: The solvers under test.

Method	Type	Order
Newton–Raphson	classical	2
Danby [4]	classical	4
Markley [5]	closed form	—
NWM9 [6]	with memory	8.8989
NWM11 [7]	with memory	10.7446
NWM9, NWM11 guarded (ours)	with memory, guarded	as published

3.2 Reference roots

Accuracy is measured against roots computed separately in `mpmath` at 50 decimal digits with a bracketing method (Anderson–Björck, with bisection as a fallback). A root is accepted only if its residual is below 10^{-40} . This code shares nothing with the solvers under test, so no solver is graded against itself.

3.3 The test grid

We test on 7,400 (e, M) points drawn in three different ways (Table 3). The even grid covers the whole domain fairly. The corner set is log-spaced, because equal steps would put almost no points near $e \rightarrow 1, M \rightarrow 0$. The RadVel set follows the eccentricity distribution of real exoplanets [11]. The even grid stops at $M = \pi$ because the solution is symmetric about it: $E(2\pi - M) = 2\pi - E(M)$.

Table 3: The three sets of test points.

Set	e	M	Points
Even grid	50 values, 0 to 0.99	100 values, 0 to π	5,000
Corner set	$1 - e$ from 10^{-1} to 10^{-4}	10^{-1} to 10^{-6}	400
RadVel set	Beta(0.867, 3.03)	uniform on $[0, 2\pi]$	2,000

When we report failure rates we use one fixed definition of the *hard corner*: $e > 0.9$ and $M < 0.1$. It holds 381 of the 7,400 points. Crossing four iterative solvers with four guesses, plus Markley once, gives 17 combinations and $17 \times 7,400 = 125,800$ solves. Each iterative solver makes $4 \times 381 = 1,524$ solves in the hard corner.

3.4 Experiments

Table 4 lists the experiments. Each has a script in `scripts/`, and all except the real-data check read their settings from a file in `configs/`. Every script writes its tables under `results/`. The verification, safeguard and real-data tables, and the raw grid results, each have a metadata file beside them recording the git commit and configuration used.

Table 4: The experiments and where their results are written.

Experiment	Question	Output
verification	Does each solver reproduce its paper’s order, and find the right root?	results/ verification/
grid bench- mark	Iterations, cost, time and failures over 125,800 solves	results/ grid_benchmark/
safeguard	Do the failures come from the memory, and does the safeguard remove them?	results/ safeguard/
propagation	How far does solver error move a fitted orbit?	results/ error_propagation/
Monte Carlo	How far does measurement noise move it?	results/ monte_carlo/
real data	Do the solvers agree on published data?	results/ real_data/

The downstream experiments use the real observation times and uncertainties of HD 164922 [12]: 401 measurements from three instruments, Keck/HIRES before and after its 2004 upgrade and the APF. The velocities are replaced by a known orbit ($P = 20.885$ d, $e = 0.35$, $\omega = 1.0$ rad, $K = 5$ m s $^{-1}$) plus Gaussian noise at the real uncertainties. A known orbit lets us measure how far each fit lands from the truth. Each solver is patched into RadVel in place of its compiled solver, and a call counter checks after every fit that the patch was actually used.

3.5 Reproducibility

The repository’s README maps each table and figure to the script and result file behind it. The test suite has 284 tests, including checks of every solver against its paper and of the safeguard’s three claims: it leaves ordinary points alone, cuts corner failures and keeps the order.

4 Results and Discussion

4.1 Verification

Table 5 shows the measured order of each iterative solver at 2000 digits. NWM9 is tested on the eight functions of its own paper, and Newton, Danby and NWM11 on the eight of the NWM11 paper. A solver passes if its measured order is within 5% of the claim, and all four pass easily. The mean measured order is at or slightly above each claim, as expected, because both papers prove the R-order as a lower bound. Single functions scatter by at most 1.1% (Figure 3).

Table 5: Measured convergence order on each paper’s eight test functions.

Solver	Claimed	Measured	Range
Newton	2.0000	2.0000	2.0000–2.0000
Danby	4.0000	4.0000	4.0000–4.0000
NWM9	8.8989	8.9262	8.8851–9.0000
NWM11	10.7446	10.7605	10.7147–10.8336

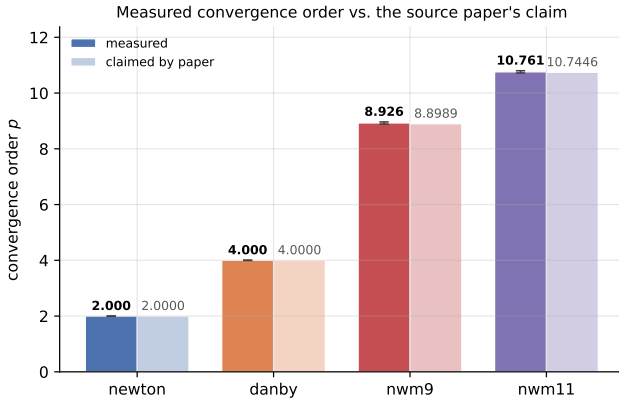


Figure 3: Measured order against each paper’s claim. Error bars show the spread over the eight test functions.

Matching the order does not prove the code is right. Both papers print nested fractions that can be read in more than one way, and several wrong readings still converge at order 8. Measuring the order would not have caught them. Each paper’s eighth-order *error constant* did. We drove each scheme one step from a point 10^{-24} away from the root and compared the measured constant with the published formula; our implementations agree to better than 10^{-20} in relative terms. One wrong grouping in NWM11’s w' missed the constant by about 60 orders of magnitude while still converging at

order 8. A separate check caught copying errors in six of NWM9’s eight test functions, where text extraction had misplaced a superscript (so xe^{x^2} came out as xe^x): the root the paper publishes did not zero our copy. As a further check, setting every self-accelerating parameter to zero gives order 8 to within 10^{-3} on all eight functions for both methods, which confirms that the extra order comes from the memory.

The right root, not just a small residual. A solver can converge to a wrong branch and still report a tiny residual. We ran each solver from $E_0 = M$ on 4,975 points against the 50-digit reference (Table 6). We call a converged solve a wrong root if it lands more than 10^{-6} from the reference. No solver returned one. The errors of order 10^{-10} come from points where f' is small: the stopping rule tests the residual, and the error in E is roughly f/f' . Where a solver does not converge it uses up its iteration limit (100 in this check) and says so. That outcome is visible and can be retried, unlike a wrong root, so we treat it as a robustness problem in Section 4.4.

Table 6: Correctness on 4,975 points against the 50-digit reference (guess $E_0 = M$, stopping tolerance 10^{-13} , at most 100 iterations).

Solver	Converged	Wrong roots	Largest $ E - E_{\text{ref}} $
Newton	99.54%	0	3.4×10^{-10}
Danby	99.72%	0	1.6×10^{-10}
NWM9	98.25%	0	2.3×10^{-10}
NWM11	99.56%	0	2.2×10^{-10}

4.2 Iterations, cost and time on the grid

Table 7 shows each solver with the starting guess that suits it best. Iterations and the median modelled cost put the with-memory methods ahead of Danby and Newton. The mean cost puts them behind Danby, and clock time puts them behind both.

Table 7: Each solver with its best guess, over all 7,400 points. Iterations, cost and time are over converged solves only. Cost is in measured sin units; Markley’s counts its (sin, cos) pair and final residual, not the square root and 2/3 power in its starter. Time is the median of single timings of each solve, guess excluded.

Solver	Guess	Mean iter.	Median cost	Time (μ s)	Conv.
Markley	—	0.00	3.34	1.7	100.00%
NWM11	canonical	1.38	6.34	2.7	99.93%
NWM9	napier	1.44	6.34	3.7	100.00%
Danby	napier	1.94	7.68	2.0	100.00%
Newton	napier	3.21	11.02	2.1	100.00%

Iterations. The with-memory methods need the fewest iterations. The median solve takes one iteration for NWM9 and NWM11, two for Danby and three for Newton.

Cost. In measured units, the median with-memory solve costs 6.34 against Danby's 7.68, about 17% less. The mean reverses this: 9.55 for NWM11 and 10.04 for NWM9 against 7.48 for Danby, 28% and 34% more. Both results follow from how the calls add up. Every solve pays one lone sin for the residual at the guess and one per iteration for the stopping test. Danby's typical solve is two iterations, each with one (sin, cos) pair, so it costs $2(2.34) + 3 = 7.68$. The typical with-memory solve is one iteration with one pair and two lone sin calls inside the step, so it costs $2.34 + 2 + 2 = 6.34$. In such a solve there is no previous iterate, so the parameters stay at the papers' starting constants ($\alpha_0 = 0.01$, and $\beta_0 = 10^{-5}$ for NWM11), and the step is only eighth order. The median saving therefore comes from the eighth-order step, not from the memory, which is also why NWM9 and NWM11 tie. The memory starts in the second iteration, which 33% of NWM11 solves and 39% of NWM9 solves need. A full with-memory iteration costs 10.62 against 3.34 for a classical one, because it adds three interpolated derivatives at 1.76 each (Figure 4), and these second iterations push the mean above Danby's. Under call counting the two kinds of iteration cost 4 and 2.

Time. On the clock the ranking reverses. Markley, Danby and Newton take 1.7 to 2.1 μ s per solve. NWM11 at its best guess takes 2.7 μ s and NWM9 3.7 μ s, 1.3 and 1.8 times as long as Danby, and NWM11 under the simple guess takes 9.5 μ s, because from $E_0 = M$ its median solve takes two iterations and the second one does the memory work. A separate timing run, 1,000 repeats of the whole call at 50 grid points, gives the same ranking: 4.5 μ s for Danby, 5.3 for NWM11 and 6.6 for NWM9. For the median solve, then, the model says the with-memory methods are cheaper and the clock says they are slower. Section 4.4.4 explains why.

4.3 The starting guess

On Kepler's equation the starting guess matters, and for NWM9 it saves more iterations than changing the method does. Figure 5 shows the mean iteration count of every solver under every guess. Napier's guess is the best for Newton, Danby and NWM9, and the canonical guess is best for NWM11, with Napier's close behind.

Changing NWM9's guess from $E_0 = M$ to Napier's formula lowers its mean from 2.37 to 1.44 iterations, a saving of 0.93, and its mean cost from 19.65 to 10.04 sin units. Moving from Danby to NWM9 with Napier's guess saves only 0.51 ($1.94 \rightarrow 1.44$). For Newton the order is reversed: the better guess saves 0.64 ($3.84 \rightarrow 3.21$), while switching to Danby saves 1.26. The guess also changes reliability. NWM9 converges on 99.43% of points from the simple guess and on all of them from Napier's.

Splitting the averages by set shows that the sets do not agree (Figure 6). On the even grid Napier's guess is the best for every solver. On the RadVel set, Napier's and the canonical guess are within 0.01 iterations of each other, and both beat $E_0 = M$. In the corner set no single guess wins: Napier's guess rescues NWM9, cutting its mean from 7.05 to 2.77, while NWM11 does best from $E_0 = M$.

The guess matters in the corner because there the root is much larger than M : on the 400 corner-set points it is a median 63 times larger. Starting from $E_0 = M$ puts the first step where $f'(E_0) = 1 - e \cos M$ is tiny (median 0.004), which is where the self-accelerating

parameter misbehaves (Section 4.4). Napier's guess starts near 0.59, above the root on 96% of these points, where the median f' is 0.18, about 46 times larger. From there NWM9 converges on all 400 points, against 91.25% from $E_0 = M$. NWM11 does not gain: it converges on 94.5% of corner-set points from Napier's guess and on 97.0% from $E_0 = M$.

Figure 7 shows where the iterations are spent. On the even grid, NWM11 never needs more than three iterations, Danby and NWM9 need more than three on fewer than 0.1% of points, and Newton needs at most four on 99.6% of points. The extra iterations grow with e and are concentrated at high e and low M .

4.4 Where and why the with-memory methods fail

4.4.1 Failures concentrate in the hard corner. Table 8 splits non-convergence by region. Of 176 failures in 125,800 solves, 155 are in the hard corner, which holds only 5% of the solves. Every failure is a solve that used up its 50 iterations. No solver crashed or returned a wrong root.

Table 8: Non-convergence by region, over all four guesses. The hard corner is $e > 0.9$, $M < 0.1$ (1,524 solves per iterative solver).

Solver	Rest of grid	Hard corner	Failures
Markley	0.000%	0.000%	0
Danby	0.000%	0.131%	2
Newton	0.018%	0.919%	19
NWM9	0.057%	4.265%	81
NWM11	0.000%	4.856%	74

Outside the corner NWM11 never fails, and NWM9 fails 16 times, all at $e > 0.9$. Inside it, NWM11 fails on 4.9% of solves and NWM9 on 4.3%, 37 and 33 times Danby's rate. Danby fails only twice there, so these ratios are rough. Figure 8 shows where each solver fails.

4.4.2 The failures come from the memory. As $e \rightarrow 1$ and $M \rightarrow 0$, the root E goes to zero, $\cos E \rightarrow 1$, and $f'(E) = 1 - e \cos E$ goes to zero. At $e = 0.9999$, $f'(0) = 10^{-4}$. A Newton-type step divides by this derivative, so a small error in the iterate becomes a large jump. All Newton-type methods feel this, which is why Newton and even Danby have a few failures here. The with-memory methods have a second problem on top of it. Take NWM11's first step. With $\alpha_k = -H_5''(s_k)/(2f'(s_k))$ its denominator is

$$f'(s_k) + \alpha_k f(s_k) = f'(s_k) \left(1 - \frac{1}{2} L_k\right), \quad L_k = \frac{f(s_k) H_5''(s_k)}{f'(s_k)^2}. \quad (6)$$

If H_5'' were the exact f'' , this step would be Halley's method. L_k is the Newton step f/f' multiplied by a second factor H_5''/f' , so it grows faster than the Newton step as f' shrinks. As L_k approaches 2 the denominator vanishes, and beyond 2 it changes sign and the step points away from the root. H_5'' makes this worse. It is fitted through the previous iterates, and after a large step those can lie far from the current point, so H_5'' need not resemble f'' there. NWM9's α_n contains the same term $-H_5''/(2f')$. For α_k , the guard of Equation (5) is exactly the condition $|L_k| \leq 1$.

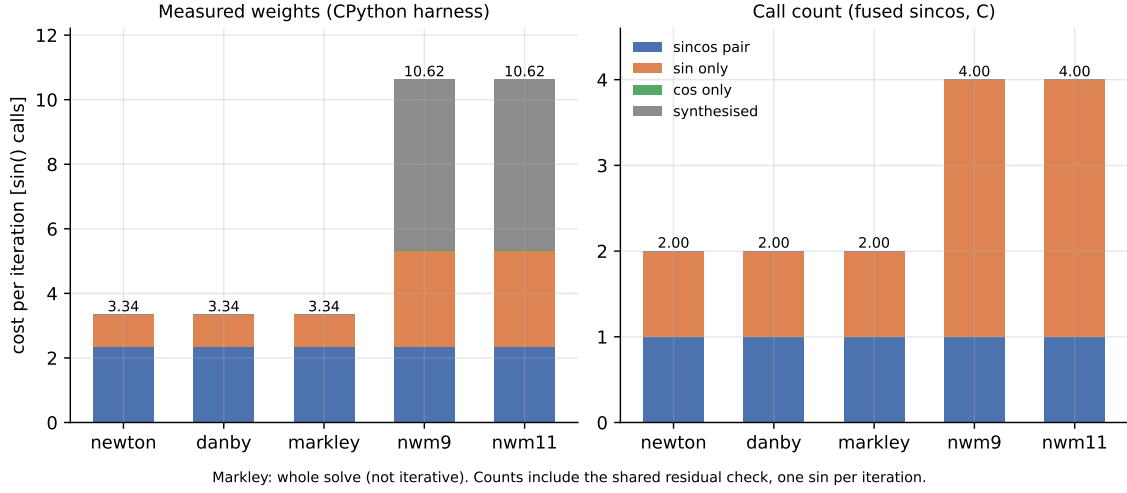


Figure 4: Cost of one iteration. Left: weights measured in our Python code. Right: call counting, as in a compiled code with a fused sincos. Most of the difference between the two panels is the interpolated derivatives, which cost 1.76 each in Python and close to nothing in compiled code.

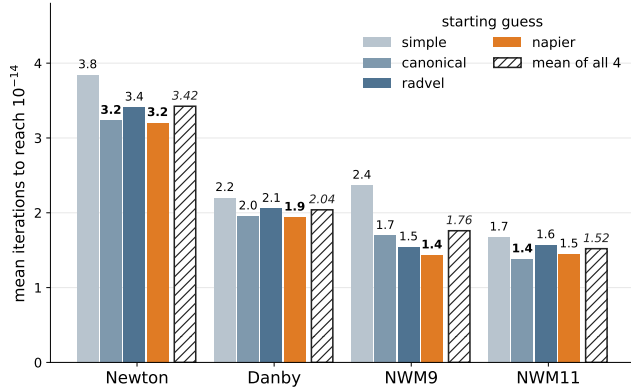


Figure 5: Mean iterations to reach $|f| \leq 10^{-14}$ for each solver and guess, over all 7,400 points. The hatched bar is the solver's mean over all four guesses.

We tested this explanation directly by running both methods with the memory switched off: every parameter fixed at zero, which gives the eighth-order parent methods. Table 10 shows the result. With the memory off, all 74 of NWM11's failed solves converge, and 79 of NWM9's 81. The memoryless methods fail instead on 9 and 18 solves, and only two of those are points where the published method also fails. The mean iteration count barely changes. The memory therefore causes nearly all of the published methods' failures, though the base methods are not free of trouble in the corner either.

4.4.3 NWM9 overshoots from a poor guess. The order scan on Kepler's equation itself (Table 9, Figure 9) shows a second weakness of NWM9, one that starts in a sub-step with no parameter. Each run starts from $E_0 = M = 0.3$ and takes ten iterations. NWM11 keeps

its order of about 10.7 all the way to $e = 0.999$. NWM9's values at $e = 0.9$ and $e \geq 0.99$ are not convergence orders. In those runs NWM9 reached the root only on the last iteration or not at all, so the estimator was fed the errors of a wandering iterate. Newton's 2.13 and 1.79 have the same cause.

The wandering starts in NWM9's second sub-step, $z = x - t [1 + u + (1 + \frac{1}{1+t}) u^2]$ with $t = f(x)/f'(x)$ and $u = f(y)/f(x)$, which contains no parameter. The saved iteration history at $e = 0.99$, $M = 0.1$ shows how. From $E_0 = M$, $f'(E_0) = 0.015$, so the Newton sub-step lands at $y \approx 6.7$ and $u \approx -63$. The u^2 term makes the bracket about 3,200, and the first iterate is $E_1 \approx 2.1 \times 10^4$. This run takes 17 more iterations to recover; from the canonical guess the iterate reaches $|E| \approx 10^{27}$ and fails. The overshoot happens in the first iteration, before any memory exists, so the guard cannot prevent it. What the memory changes is the recovery. The next parameter is fitted through iterates that lie far apart, so it no longer describes f near the root, and the run keeps wandering. With the memory off, 79 of NWM9's 81 failed solves converge. A better guess avoids the overshoot altogether: from Napier's guess NWM9 converges on all 7,400 grid points (Section 4.3).

Table 9: Order measured on Kepler's equation at $M = 0.3$ from $E_0 = M$, ten iterations at 2000 digits. A dash means the estimate could not be resolved. NWM9's entries at $e = 0.9$ and $e \geq 0.99$, and Newton's at $e \geq 0.99$, come from runs that had not converged (see text).

e	0.3	0.7	0.9	0.95	0.99	0.995	0.999
Newton	2.00	2.00	2.00	2.00	2.13	—	1.79
Danby	4.00	4.00	4.00	4.00	4.00	4.00	4.00
NWM9	8.90	8.90	2.82	8.90	4.48	0.72	0.36
NWM11	10.72	10.76	10.88	10.84	10.73	10.74	10.74

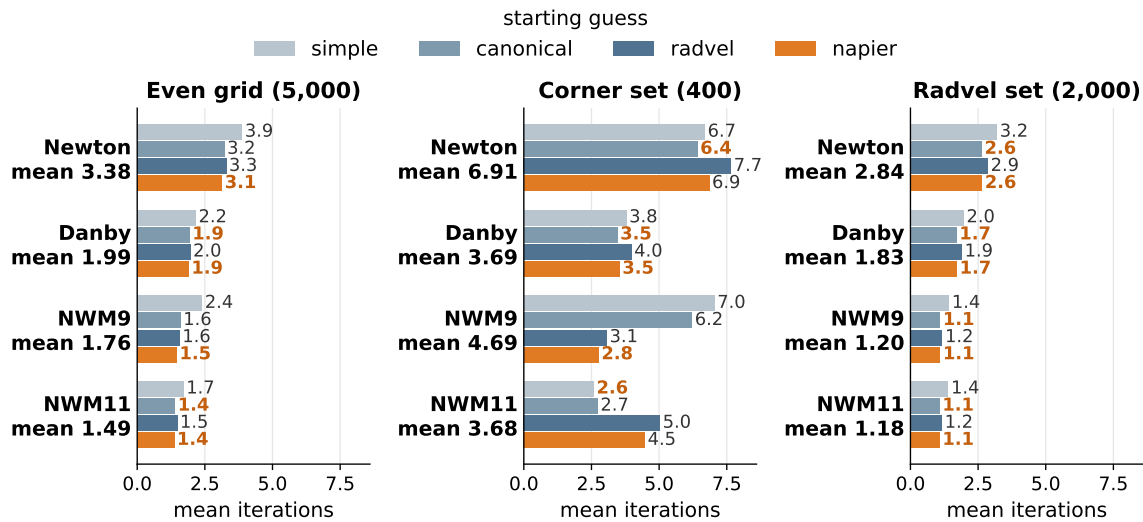


Figure 6: Mean iterations in each set of test points. Orange marks the fewest iterations for that solver in that set. The mean under each solver is its average over all four guesses. Only converged solves are counted.

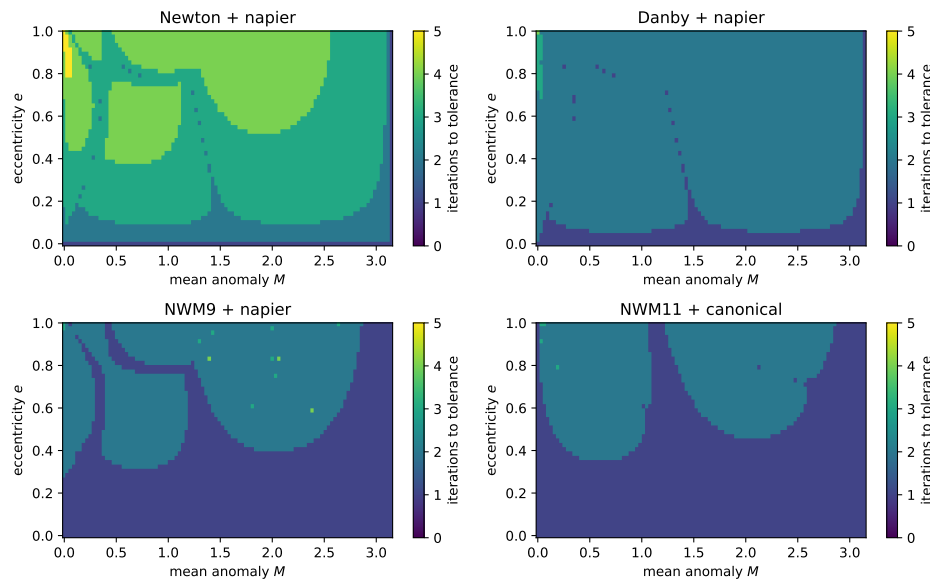


Figure 7: Iterations to tolerance on the even grid, each solver with its best guess. All four solvers are cheap almost everywhere; the cost is concentrated near $e \rightarrow 1, M \rightarrow 0$.

4.4.4 Why the cost model misreads the clock. Section 4.2 found the with-memory methods cheaper in the cost model but slower on the clock. The cost argument for these methods assumes that arithmetic is free and that one call returns both $\sin E$ and $\cos E$ for about the price of one. Neither holds in Python. CPython has no fused sincos, so a pair costs 2.34, against 2.04 for a sin and a cos timed separately, and an interpolated derivative, a few interpreted operations, costs 1.76. Our measured weights include both, yet they still rate a one-iteration with-memory solve below a two-iteration Danby solve

(6.34 against 7.68). On the clock the one-iteration solve takes a median $2.7 \mu\text{s}$ for NWM11 and $3.6 \mu\text{s}$ for NWM9, against $2.1 \mu\text{s}$ for the two-iteration Danby solve. That solve uses no interpolated derivative, so the gap is the interpreted arithmetic of the three-step formula itself, which the model does not count (Figure 10). Compiled code would shrink that overhead and could fuse the pair, but even then our call counts put the with-memory methods only level with Danby on a median solve.

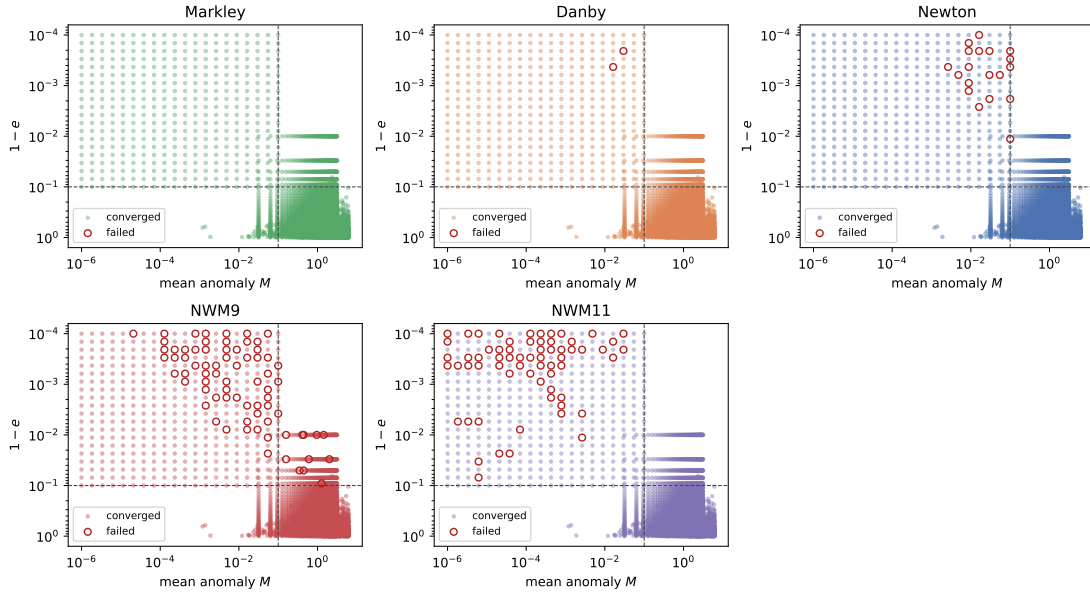


Figure 8: Converged and failed solves for each solver, all guesses together. The vertical axis is $1 - e$ on a log scale, so the top of each panel is $e \rightarrow 1$; dashed lines mark the hard corner. Almost every failure lies inside it.

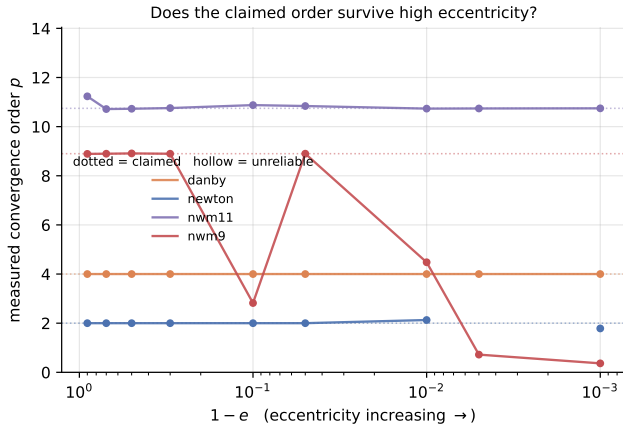


Figure 9: Measured order against eccentricity. Dotted lines mark each method's claimed order; hollow markers are estimates that could not be resolved.

4.5 The safeguard repairs NWM11

Switching the memory off everywhere removes most failures but gives up the order the papers prove. The safeguard of Section 2.4 keeps the memory where it is safe. Table 10 and Figure 11 compare the three versions of each method on the full grid.

NWM11. The guard cuts NWM11's failures from 74 to 3. That is fewer than switching the memory off (9), and close to Danby (2). The guarded version converges on all 9 solves that fail with the memory off, so once its unsafe steps are removed the memory makes NWM11 more robust in the corner. With the canonical guess,

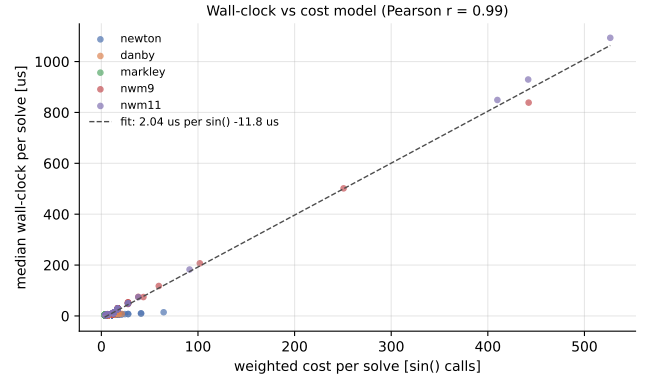


Figure 10: Wall-clock time against modelled cost, one point per timed solve. Over all points $r = 0.99$, but that figure is carried by a few very long solves (up to 50 iterations at $e > 0.93$). Over typical solves (88% of points, cost at most 15) the correlation falls to $r = 0.42$.

guarded NWM11 converges on all 7,400 points in 1.38 iterations on average, against Danby's best of 1.94. The three that remain are at $e \geq 0.9995$ and $M \leq 2.1 \times 10^{-5}$. Outside the hard corner the guard changes little: guarded NWM11 takes the same number of iterations as the published version on 99.6% of the 28,076 solves outside it, and its largest error there is unchanged at 5.7×10^{-14} . The guard also keeps the order. At 2000 digits, guarded NWM11 measures 10.76 on the paper's test functions, the same as the published version, and it matches the published version at every eccentricity in Table 9, up to 10.74 at $e = 0.999$.

Table 10: Published, memory-off and guarded versions on all 7,400 points and four guesses (29,600 solves each). Mean iterations count converged solves only.

Version	Failures	Corner rate	Mean iter.
Danby (reference)	2	0.13%	2.04
NWM9 published	81	4.27%	1.76
NWM9 memory off	18	0.85%	1.74
NWM9 guarded	17	0.85%	1.75
NWM11 published	74	4.86%	1.52
NWM11 memory off	9	0.59%	1.51
NWM11 guarded	3	0.20%	1.53

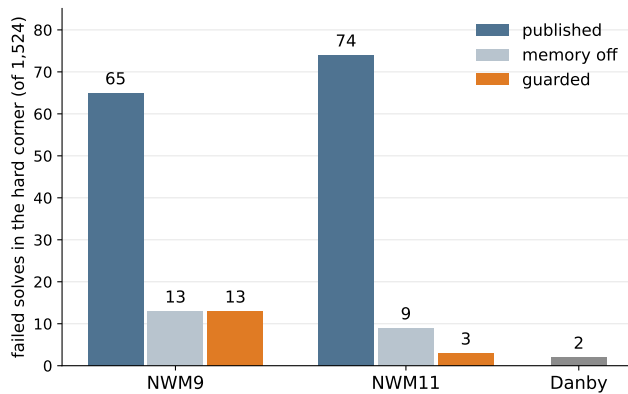


Figure 11: Failed solves in the hard corner (out of 1,524) for the published, memory-off and guarded versions of each method, with Danby for reference.

NWM9. The guard cuts NWM9’s failures from 81 to 17, about as many as switching the memory off leaves (18). The remaining failures are mostly different solves: the published NWM9 converges on 11 of the 17, and the memory-off version on 15. All 17 start from the simple or canonical guess at $e > 0.94$, where NWM9’s second sub-step can throw the first iterate far from the root (Section 4.4.3). That sub-step has no parameter, so the guard does not act on it. The guard keeps NWM9’s order on its test functions (8.93), and with Napier’s guess both the published and the guarded NWM9 converge on every point.

The guard is the improvement we propose to the base papers: a test that costs two multiplications per parameter, needs no new evaluations, and leaves the method unchanged wherever the papers’ analysis applies. On Kepler’s equation it brings NWM11 close to Danby’s robustness (3 failures against 2) while keeping its lower iteration count.

4.6 Does solver error matter downstream?

We traced solver error from E through the true anomaly ν and the radial velocity v_r into the fitted orbit. The question is how large the solver’s effect is compared with the uncertainty that is already in the data. For each solver we refit the same injected dataset at six tolerances, from 10^{-4} to 10^{-14} , and measure how far each

fitted parameter moves from that solver’s own fit at 10^{-14} . Table 11 compares the largest such shift with the spread caused by noise across 50 refits with fresh noise, and with RadVel’s own MCMC posterior width.

Table 11: Error budget. Solver shift: the largest shift over all five solvers and all tolerances (in every case Newton at 10^{-4}). Noise: standard deviation over 50 refits with Danby at 10^{-14} , each with fresh noise. MCMC: RadVel posterior width from one MCMC run capped at 2000 steps.

Param.	Solver	Noise	MCMC	Noise/solver
P (d)	1.0×10^{-8}	5.0×10^{-4}	6.4×10^{-4}	51,000
e	1.1×10^{-5}	1.65×10^{-2}	1.74×10^{-2}	1,500
ω	2.3×10^{-5}	5.03×10^{-2}	5.53×10^{-2}	2,200
K	3.5×10^{-5}	9.93×10^{-2}	1.02×10^{-1}	2,800

Even in the worst case, Newton at a tolerance of 10^{-4} , the solver moves each parameter at least 1,490 times less than the noise does. The sizes involved explain why. An error δE changes the velocity by at most $K (dv/dE) \delta E$, and for this orbit $dv/dE \leq 1.44$, so an error of 10^{-4} in E moves v_r by at most $7 \times 10^{-4} \text{ m s}^{-1}$. The measurement uncertainties are 0.8 to 3.0 m s^{-1} .

Only Newton at 10^{-4} shows a clear solver effect, 6.4×10^{-4} posterior widths in e . Newton’s last step can stop with $|f|$ just under the tolerance, which leaves an error of about 10^{-4} in E ; a fourth-order or higher step that passes the same test lands many orders of magnitude below it. Every other combination stays at or below about 10^{-5} posterior widths, and this floor does not shrink as the tolerance tightens. It comes from the optimiser’s own stopping precision, not from the solver: at 10^{-14} the five solvers’ fits still differ from each other by up to 1.2×10^{-7} in e , about 7×10^{-6} posterior widths. Below 10^{-4} we can therefore only bound the solver’s effect, not measure it (Figure 12). The shift is zero at 10^{-14} by construction, and Markley, which has no tolerance, gives the same fit at every setting.

The spread across the 50 noisy refits is smaller than the MCMC posterior width by 3% for K , 5% for e , 9% for ω and 21% for P . The two estimates of the noise are computed independently, so their agreement checks both.

Cost of a whole fit. Table 12 shows the time of a full RadVel fit with each solver. All of our solvers are 20 to 32 times slower than RadVel’s compiled one. Most of that gap comes from calling a scalar Python solver once per point, where RadVel works on whole arrays, so these numbers compare our solvers with each other, not with production code. Newton and Markley take 0.89 and 1.01 s, NWM11 1.29 s and NWM9 1.52 s. Danby’s six fits range from 0.91 to 1.88 s, so its median of 1.32 s is rough. Each slowdown is a median of ratios to a RadVel fit timed alongside, so it need not follow the order of the fit times, and Danby’s comes out below NWM11’s. With one timing per fit, the only clear result is that every NWM9 fit took longer than every Newton or Markley fit.

Real data. The injection study uses one orbit, so we also refitted three published datasets with each solver: K2-131 [13] (70 measurements), HD 164922 [12] (401) and K2-24 [14, 15] (32). The period,

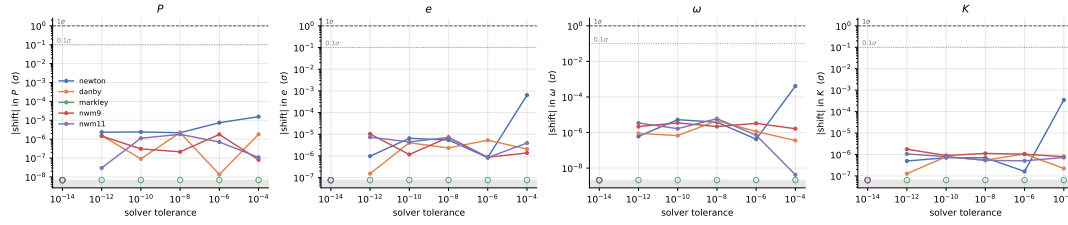


Figure 12: Parameter shift against solver tolerance, in units of the MCMC posterior width. Only Newton at 10^{-4} rises clearly above the floor set by the optimiser.

Table 12: Time of one full RadVel fit (about 1.3×10^5 solves), median over six tolerances. Each fit was timed once.

Solver	Fit (s)	vs. RadVel	μ s per solve
Newton	0.89	20.0×	7.0
Markley	1.01	21.9×	7.8
NWM11	1.29	28.4×	9.4
Danby	1.32	23.9×	9.7
NWM9	1.52	31.8×	10.8

and the transit time where there is one, were held fixed as in RadVel’s own examples, because with them free the sparse data let the fit jump to an alias of the period. The five solvers agree on the fitted eccentricity to within 6.4×10^{-7} for K2-131, 7.1×10^{-9} for HD 164922 and 6.9×10^{-9} for K2-24. These fitted values are not orbit measurements: HD 164922 and K2-24 are multi-planet systems fitted with one planet. The check only shows that the solvers agree with each other on real data, as they do on the injected orbit.

4.7 Summary of findings

Table 13 collects the results.

Table 13: Findings by question.

Question	Answer
Order	Both papers’ claims hold (8.93 and 10.76 measured).
Iterations	With-memory methods need the fewest (1.38 vs. 1.94 for Danby).
Model cost	Median solve 17% cheaper than Danby, from the eighth-order parent, not the memory; the mean solve costs more.
Clock time	Slower than Danby in Python: 1.4–1.8× per solve; per fit, NWM9 is slowest.
Corner	Published versions fail 33–37× as often as Danby.
Cause	The self-accelerating parameter, where $f' \approx 0$.
Fix	Guarded NWM11: 74 → 3 failures, same order and iterations.
Guess	Napier’s is best for Newton, Danby and NWM9; for NWM9 it saves more than changing method.
Downstream	Even Newton at 10^{-4} moves the fit about 1,500× less than noise.

5 Conclusion and Future Work

We reimplemented two recent with-memory methods, confirmed that they reach their published orders, and tested them on Kepler’s equation against the methods astronomers already use. Their high order is real: they need fewer iterations than any other iterative method, and with our measured weights their median solve is 17% cheaper than Danby’s. That saving comes from one-iteration solves, where the memory is not yet used. It does not hold on average: a third or more of solves need a second iteration, which uses the memory and costs 10.62 units against 3.34 for a classical one. In Python the saving does not show up on the clock either.

Their weakness is the corner where $f' \approx 0$. There the self-accelerating parameter, derived for iterates already near the root, becomes as large as the derivative it corrects and throws the step far from the root. Switching the memory off showed that this, not the base method, causes most failures. A safeguard that keeps the parameter only while its correction is less than half of $|f'|$ removes almost all of NWM11’s failures and leaves its order and iteration count unchanged. With this change NWM11 fails on 3 of 29,600 solves against Danby’s 2, and still needs fewer iterations (1.38 against 1.94, each at its best guess). The guard helps NWM9 less (81 to 17 failures), because NWM9’s overshoot happens in a sub-step that has no parameter. A better starting guess avoids it: from Napier’s guess NWM9 converges on every grid point, and changing its guess from $E_0 = M$ to Napier’s saves it more iterations than switching from Danby to NWM9 does.

Downstream, the solver does not change the fitted orbit in any way that matters: even Newton stopped at 10^{-4} moves it about 1,500 times less than the measurement noise does. A new solver therefore has to win on speed. In Python it does not, so Danby or Markley remain the practical choice. Our call counts do not promise a win in compiled code either: with one call per (sin, cos) pair and free arithmetic, the median solve costs five calls for Danby, NWM9 and NWM11 alike, and the mean is higher for guarded NWM11 (6.3 against 4.9 for Danby).

5.1 Limitations

- Python only.** All timings are in CPython, which has no fused sincos and makes arithmetic expensive. The expensive arithmetic works against the with-memory methods, whose steps do more of it.
- One machine.** The cost weights were measured on one machine, under CPython 3.14, while the benchmarks ran under CPython 3.12 on the same machine; the weights may

differ elsewhere. Each RadVel fit was timed once, and the six fits for one solver vary by up to 1.0 s (Danby’s took 0.91 to 1.88 s), so per-fit times separate only NWM9 from Newton and Markley.

- (3) **The guard constant.** We set $c = 1/2$ from the guard’s own algebra (Section 2.4), did not tune it, and report no other value. The guard has been tested only on our grid.
- (4) **NWM9’s remaining failures.** The guard leaves 17 NWM9 failures, all from the simple or canonical guess at $e > 0.94$. It does not act on NWM9’s second sub-step, which has no parameter.
- (5) **One injected orbit.** The downstream study uses one orbit with $e = 0.35$, and its posterior widths come from one short RadVel MCMC run, capped at 2000 steps. The margin over noise is large, but more eccentric orbits were not tested.
- (6) **The incumbent.** RadVel’s compiled Danby, with its own guess and a 10^{-12} tolerance, was not run under our cost counting. Our “Danby” is our Python version at 10^{-14} .

5.2 Future work

The most direct next step is a C or Cython port with a fused sincos, which would show whether the with-memory methods can win on time at all. Second, NWM10, the one-parameter method from the same paper as NWM11, would be the right control for measuring what the second parameter adds; NWM9 comes from a different paper with a different base method. Third, the guard constant should be checked on new test points and on other equations with small derivatives, to see whether the safeguard helps with-memory methods in general. Finally, injecting more eccentric orbits would test the downstream result where the solver has the most chance to matter.

References

- [1] B. J. Fulton, E. A. Petigura, S. Blunt, and E. Sinukoff. RadVel: The Radial Velocity Modeling Toolkit. *Publications of the Astronomical Society of the Pacific*, 130(986):044504, 2018. doi:10.1088/1538-3873/aaaaa8.
- [2] California Planet Search. RadVel source code, `src/kepler.c` and `radvel/kepler.py`. <https://github.com/California-Planet-Search/radvel>.
- [3] C. D. Murray and S. F. Dermott. *Solar System Dynamics*. Cambridge University Press, 1999.
- [4] J. M. A. Danby. The solution of Kepler’s equation, III. *Celestial Mechanics*, 40(3–4):303–312, 1987. doi:10.1007/BF01235847.
- [5] F. L. Markley. Kepler equation solver. *Celestial Mechanics and Dynamical Astronomy*, 63(1):101–111, 1995. doi:10.1007/BF00691917.
- [6] S. K. Mittal, S. Panday, and L. Jäntschi. Enhanced ninth-order memory-based iterative technique for efficiently solving nonlinear equations. *Mathematics*, 12(22):3490, 2024. doi:10.3390/math12223490.
- [7] S. K. Mittal, S. Panday, L. Jäntschi, and L. C. Bolundut. Two novel efficient memory-based multi-point iterative methods for solving nonlinear equations. *AIMS Mathematics*, 10(3):5421–5443, 2025. doi:10.3934/math.2025250.
- [8] G. Matthies, M. Salimi, S. Sharifi, and J. L. Varona. An optimal three-point eighth-order iterative method without memory for solving nonlinear equations with its dynamics. *Japan Journal of Industrial and Applied Mathematics*, 33:751–766, 2016. arXiv:1602.07026.
- [9] O. S. Solaiman and I. Hashim. Optimal eighth-order solver for nonlinear equations with applications in chemical engineering. *Intelligent Automation & Soft Computing*, 27(2):379–390, 2021. doi:10.32604/iasc.2021.015285.
- [10] K. J. Napier. Improved initial guesses for numerical solutions of Kepler’s equation. arXiv:2411.15374, 2024.
- [11] D. M. Kipping. Parametrizing the exoplanet eccentricity distribution with the Beta distribution. *Monthly Notices of the Royal Astronomical Society*, 434:L51–L55, 2013. doi:10.1093/mnras/slt075.
- [12] B. J. Fulton et al. Three temperate Neptunes orbiting nearby stars. *The Astrophysical Journal*, 830(1):46, 2016. doi:10.3847/0004-637X/830/1/46.
- [13] F. Dai et al. The discovery and mass measurement of a new ultra-short-period planet: K2-131b. *The Astronomical Journal*, 154(6):226, 2017. doi:10.3847/1538-3881/aa9065.
- [14] E. A. Petigura et al. Two transiting low density sub-Saturns from K2. *The Astrophysical Journal*, 818(1):36, 2016.
- [15] E. A. Petigura et al. Dynamics and formation of the near-resonant K2-24 system. *The Astronomical Journal*, 156:89, 2018. doi:10.3847/1538-3881/aaceac.
- [16] F. Johansson et al. mpmath: a Python library for arbitrary-precision floating-point arithmetic, version 1.4.1. <https://mpmath.org>.
- [17] C. R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- [18] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.
- [19] W. McKinney. Data structures for statistical computing in Python. In *Proc. 9th Python in Science Conference*, pages 56–61, 2010.
- [20] J. D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [21] D. Foreman-Mackey, D. W. Hogg, D. Lang and J. Goodman. emcee: the MCMC hammer. *Publications of the Astronomical Society of the Pacific*, 125:306–312, 2013.